



E2E_TEST_SPECIFICATION

HELIXCODE END-TO-END TEST SPECIFICATION

Status: 90 Test Cases Missing

Framework: Ready (infrastructure exists)

Priority: CRITICAL (blocking production readiness)

Implementation: Week 2-3 of Phase 1

E2E TEST COVERAGE MATRIX

Current Status: 10/100 Test Cases Implemented

Category	Planned	Implemented	Missing	Priority	Complexity
Core Workflows	25	0	25	CRITICAL	Medium
Integration Tests	30	0	30	CRITICAL	High
Distributed Systems	20	0	20	HIGH	High
Platform-Specific	15	0	15	MEDIUM	Medium
Performance	10	10	0	Complete	-
TOTAL	100	10	90	CRITICAL	-

CORE WORKFLOW TESTS (25 Tests)

1. User Authentication & Authorization (3 Tests)

TEST-E2E-001: User Registration Flow

```
func TestUserRegistration(t *testing.T) {  
    // Given: Clean database, registration endpoint available
```

```
// When: User submits valid registration data
// Then: User account created, confirmation sent
```

Steps:

1. POST /api/v1/auth/register with valid data
2. Verify 201 Created response
3. Check confirmation email sent
4. Confirm email via token
5. Verify user can login

Assertions:

- HTTP status codes correct
- User record created in database
- Password properly hashed
- JWT token generated on login
- Rate limiting enforced

```
}
```

TEST-E2E-002: User Login/Logout Flow

```
func TestUserLoginLogout(t *testing.T) {
    // Given: Existing user account
    // When: User attempts login/logout
    // Then: Session managed correctly
```

Steps:

1. POST /api/v1/auth/login with credentials
2. Verify JWT token returned
3. Use token **for** authenticated request
4. POST /api/v1/auth/logout
5. Verify token invalidated

Assertions:

- Login returns valid JWT
- Token works **for** API access
- Logout invalidates token
- Session cleanup performed

```
}
```

TEST-E2E-003: Role-Based Access Control

```
func TestRoleBasedAccess(t *testing.T) {
    // Given: Users with different roles (admin, user, guest)
    // When: Users attempt various operations
```

```
// Then: Permissions enforced correctly
```

Steps:

1. Create users with different roles
2. Attempt admin-only operations
3. Attempt user operations
4. Attempt guest operations
5. Verify proper **error** responses

Assertions:

- Admin can access all endpoints
- Users have limited access
- Guests have read-only access
- Proper HTTP **403** responses

```
}
```

2. Project Lifecycle Management (4 Tests)

TEST-E2E-004: Project Creation Flow

```
func TestProjectCreation(t *testing.T) {  
    // Given: Authenticated user  
    // When: User creates new project  
    // Then: Project created with proper structure
```

Steps:

1. POST /api/v1/projects with project data
2. Verify project directory created
3. Check project metadata stored
4. Validate initial configuration
5. Verify user permissions set

Assertions:

- Project ID generated
- Directory structure created
- Metadata properly stored
- User has owner permissions
- Default configuration applied

```
}
```

TEST-E2E-005: Project Import/Export

```
func TestProjectImportExport(t *testing.T) {  
    // Given: Existing project or external project
```

```
// When: User imports/exports project
// Then: Project transferred correctly
```

Steps:

1. Export existing project
2. Verify archive contents
3. Import project to **new** location
4. Compare imported vs original
5. Verify functionality preserved

Assertions:

- Export creates valid archive
- Import recreates project correctly
- Configuration preserved
- No data loss occurs

```
}
```

TEST-E2E-006: Project Deletion and Recovery

```
func TestProjectDeletionRecovery(t *testing.T) {
    // Given: Existing project
    // When: User deletes project
    // Then: Project properly archived/recovered
```

Steps:

1. Create test project with data
2. Delete project via API
3. Verify soft deletion (archived)
4. Attempt recovery
5. Verify permanent deletion option

Assertions:

- Soft deletion by **default**
- Recovery possible within timeframe
- Hard deletion available
- Data cleanup performed

```
}
```

TEST-E2E-007: Project Sharing and Collaboration

```
func TestProjectCollaboration(t *testing.T) {
    // Given: Project owned by user A
    // When: User A shares with user B
    // Then: Collaboration works correctly
```

Steps:

1. User A creates project
2. User A invites user B
3. User B accepts invitation
4. Both users **make** changes
5. Verify conflict resolution

Assertions:

- Invitations sent/received
- Permissions granted correctly
- Changes synchronized
- Conflict handling works

}

3. File Operations & Workspace (3 Tests)

TEST-E2E-008: File CRUD Operations

```
func TestFileCRUD(t *testing.T) {  
    // Given: Active project workspace  
    // When: User performs file operations  
    // Then: Operations succeed correctly
```

Steps:

1. Create **new** file via API
2. Read file contents
3. Update file contents
4. Delete file
5. Verify file history

Assertions:

- All CRUD operations work
- File permissions respected
- History tracked
- Changes persisted

}

TEST-E2E-009: Directory Management

```
func TestDirectoryManagement(t *testing.T) {  
    // Given: Project workspace  
    // When: User manages directories  
    // Then: Structure maintained correctly
```

Steps:

1. Create nested directory structure
2. Move files between directories
3. Rename directories
4. Delete directories
5. Verify integrity

Assertions:

- Directory creation works
- File moves preserved
- Renames propagate correctly
- Deletions are safe

}

TEST-E2E-010: File Synchronization

```
func TestFileSynchronization(t *testing.T) {  
    // Given: Multiple clients/workers  
    // When: Files modified concurrently  
    // Then: Synchronization handles conflicts
```

Steps:

1. Create file on client A
2. Modify file on client B
3. Both clients sync
4. Verify conflict resolution
5. Check version history

Assertions:

- Conflicts detected
- Resolution strategy applied
- History preserved
- No data loss

}

4. Code Generation & Editing (5 Tests)

TEST-E2E-011: AI Code Generation

```
func TestAICodeGeneration(t *testing.T) {  
    // Given: Active LLM provider  
    // When: User requests code generation  
    // Then: Code generated and integrated
```

Steps:

1. Submit generation request
2. Verify AI provider called
3. Check generated code quality
4. Apply code to project
5. Validate functionality

Assertions:

- Generation request processed
- Provider integration works
- Code quality acceptable
- Integration successful

}

TEST-E2E-012: Multi-File Editing

```
func TestMultiFileEditing(t *testing.T) {  
    // Given: Project with multiple files  
    // When: User performs multi-file edit  
    // Then: All files updated correctly
```

Steps:

1. Select multiple files
2. Define edit operation
3. Apply changes atomically
4. Verify all files updated
5. Check rollback capability

Assertions:

- Atomic operation performed
- All files updated
- Rollback possible
- Changes tracked

}

TEST-E2E-013: Code Refactoring

```
func TestCodeRefactoring(t *testing.T) {  
    // Given: Existing codebase  
    // When: User initiates refactoring  
    // Then: Code refactored safely
```

Steps:

1. Select refactoring **type**
2. Define refactoring parameters
3. Preview changes
4. Apply refactoring
5. Verify functionality preserved

Assertions:

- Refactoring preview shown
- Changes applied correctly
- Functionality preserved
- Tests still pass

}

TEST-E2E-014: Code Review Workflow

```
func TestCodeReviewWorkflow(t *testing.T) {  
    // Given: Code changes ready for review  
    // When: Review process initiated  
    // Then: Review completed successfully
```

Steps:

1. Create code changes
2. Submit **for** review
3. Reviewer adds comments
4. Address feedback
5. Approve and merge

Assertions:

- Review request created
- Comments added correctly
- Feedback addressed
- Approval process works

}

TEST-E2E-015: Version Control Integration

```
func TestVersionControlIntegration(t *testing.T) {  
    // Given: Project with Git repository  
    // When: User performs VCS operations  
    // Then: Git integration works correctly
```

Steps:

1. Initialize Git repository
2. Make changes to files

3. Stage and commit changes
4. Create and merge branches
5. Handle conflicts

Assertions:

- Git operations succeed
- History preserved
- Conflicts handled
- Remote sync works

}

5. Build & Test Automation (4 Tests)

TEST-E2E-016: Automated Build Process

```
func TestAutomatedBuild(t *testing.T) {  
    // Given: Project with build configuration  
    // When: Build triggered automatically  
    // Then: Build completes successfully
```

Steps:

1. Configure build settings
2. Trigger build via API
3. Monitor build progress
4. Check build artifacts
5. Verify deployment ready

Assertions:

- Build configuration valid
- Build triggers correctly
- Progress monitoring works
- Artifacts generated

}

TEST-E2E-017: Test Suite Execution

```
func TestTestSuiteExecution(t *testing.T) {  
    // Given: Project with test suite  
    // When: Tests executed automatically  
    // Then: Results reported correctly
```

Steps:

1. Configure test settings
2. Trigger test execution

3. Monitor test progress
4. Review test results
5. Handle test failures

Assertions:

- Tests execute correctly
- Results accurate
- Failures handled
- Reporting works

}

TEST-E2E-018: Continuous Integration

```
func TestContinuousIntegration(t *testing.T) {  
    // Given: CI/CD pipeline configured  
    // When: Code changes pushed  
    // Then: Pipeline executes correctly
```

Steps:

1. Push code changes
2. Trigger CI pipeline
3. Run build and tests
4. Deploy **if** successful
5. Notify stakeholders

Assertions:

- Pipeline triggered
- All stages execute
- Deployment successful
- Notifications sent

}

TEST-E2E-019: Deployment Automation

```
func TestDeploymentAutomation(t *testing.T) {  
    // Given: Application ready for deployment  
    // When: Deployment triggered  
    // Then: Application deployed correctly
```

Steps:

1. Configure deployment settings
2. Trigger deployment
3. Monitor deployment progress
4. Verify application running

5. Test functionality

Assertions:

- Deployment configured
- Process executes
- Application healthy
- Functionality verified

}

6. Debugging Sessions (3 Tests)

TEST-E2E-020: Debug Session Initialization

```
func TestDebugSessionInit(t *testing.T) {  
    // Given: Application ready for debugging  
    // When: Debug session started  
    // Then: Session established correctly
```

Steps:

1. Select debug configuration
2. Start debug session
3. Verify connection established
4. Check breakpoints set
5. Validate environment ready

Assertions:

- Configuration valid
- Session established
- Breakpoints active
- Environment ready

}

TEST-E2E-021: Breakpoint Management

```
func TestBreakpointManagement(t *testing.T) {  
    // Given: Active debug session  
    // When: Breakpoints added/removed  
    // Then: Breakpoints managed correctly
```

Steps:

1. Set breakpoints in code
2. Run to breakpoint
3. Inspect variables
4. Continue execution

5. Remove breakpoints

Assertions:

- Breakpoints set correctly
- Execution stops
- Variables inspectable
- Continue works

}

TEST-E2E-022: Error Handling and Recovery

```
func TestErrorHandlingRecovery(t *testing.T) {  
    // Given: Application with errors  
    // When: Errors occur during execution  
    // Then: Errors handled and reported
```

Steps:

1. Introduce **error** condition
2. Execute problematic code
3. Catch and analyze **error**
4. Apply fix
5. Verify resolution

Assertions:

- Errors caught
- Analysis accurate
- Fixes applied
- Resolution verified

}

7. Configuration Management (3 Tests)

TEST-E2E-023: Configuration Loading

```
func TestConfigurationLoading(t *testing.T) {  
    // Given: Configuration files present  
    // When: Application loads configuration  
    // Then: Settings applied correctly
```

Steps:

1. Create configuration files
2. Start application
3. Verify settings loaded
4. Check override precedence

5. Validate final configuration

Assertions:

- Files parsed correctly
- Settings applied
- Overrides work
- Validation passes

}

TEST-E2E-024: Runtime Configuration Updates

```
func TestRuntimeConfigUpdates(t *testing.T) {  
    // Given: Running application  
    // When: Configuration updated at runtime  
    // Then: Changes applied dynamically
```

Steps:

1. Start with initial config
2. Modify configuration
3. Apply changes
4. Verify updates active
5. Check persistence

Assertions:

- Changes detected
- Updates applied
- No restart needed
- Changes persist

}

TEST-E2E-025: Environment-Specific Configuration

```
func TestEnvironmentSpecificConfig(t *testing.T) {  
    // Given: Multiple environments (dev, staging, prod)  
    // When: Application runs in different environments  
    // Then: Appropriate configuration used
```

Steps:

1. Set environment variable
2. Load configuration
3. Verify correct settings
4. Test multiple environments
5. Validate secrets handling

Assertions:

- Environment detected
- Correct config loaded
- Secrets handled safely
- No conflicts

}

INTEGRATION TESTS (30 Tests)

1. LLM Provider Integration (6 Tests)

TEST-E2E-026: Provider Switching

```
func TestLLMProviderSwitching(t *testing.T) {  
    // Given: Multiple LLM providers configured  
    // When: User switches between providers  
    // Then: Switching works seamlessly
```

Steps:

1. Configure multiple providers
2. Test with first provider
3. Switch to second provider
4. Verify functionality
5. Check fallback behavior

Assertions:

- Providers **switch** correctly
- Functionality preserved
- Fallbacks work
- No data loss

}

TEST-E2E-027: Provider Fallback

```
func TestLLMProviderFallback(t *testing.T) {  
    // Given: Primary provider fails  
    // When: Fallback mechanism activated  
    // Then: Secondary provider used
```

Steps:

1. Configure primary/secondary providers
2. Simulate primary failure
3. Verify fallback triggered

4. Test functionality
5. Monitor fallback metrics

Assertions:

- Failure detected
- Fallback activated
- Functionality maintained
- Metrics recorded

}

TEST-E2E-028: Multi-Provider Load Balancing

```
func TestMultiProviderLoadBalancing(t *testing.T) {  
    // Given: Multiple providers with load balancing  
    // When: High volume of requests  
    // Then: Load distributed correctly
```

Steps:

1. Configure load balancing
2. Generate high request volume
3. Monitor distribution
4. Verify performance
5. Check health monitoring

Assertions:

- Load distributed
- Performance optimal
- Health checks work
- Balancing effective

}

TEST-E2E-029: Provider Health Monitoring

```
func TestProviderHealthMonitoring(t *testing.T) {  
    // Given: LLM providers configured  
    // When: Health checks performed  
    // Then: Health status accurate
```

Steps:

1. Configure health checks
2. Monitor healthy providers
3. Simulate provider issues
4. Verify health updates
5. Test recovery detection

Assertions:

- Health checks run
- Status accurate
- Issues detected
- Recovery noticed

}

TEST-E2E-030: Provider Rate Limiting

```
func TestProviderRateLimiting(t *testing.T) {  
    // Given: Providers with rate limits  
    // When: Rate limits exceeded  
    // Then: Limiting enforced correctly
```

Steps:

1. Configure rate limits
2. Make requests within limits
3. Exceed rate limits
4. Verify enforcement
5. Test recovery

Assertions:

- Limits enforced
- Proper responses
- Recovery works
- Metrics accurate

}

TEST-E2E-031: Provider Authentication

```
func TestProviderAuthentication(t *testing.T) {  
    // Given: Providers requiring authentication  
    // When: Authentication configured  
    // Then: Auth works correctly
```

Steps:

1. Configure API keys/tokens
2. Test authentication
3. Verify secure storage
4. Test key rotation
5. Check **error** handling

Assertions:

- Auth configured
- Keys secured
- Rotation works
- Errors handled

```
}
```

2. Database Operations (5 Tests)

TEST-E2E-032: Database Connection Management

```
func TestDatabaseConnectionManagement(t *testing.T) {  
    // Given: Database configured  
    // When: Connections established/managed  
    // Then: Connections handled efficiently  
  
    Steps:  
    1. Configure database connection  
    2. Establish connections  
    3. Test connection pooling  
    4. Verify timeout handling  
    5. Check reconnection  
  
    Assertions:  
    - Connections established  
    - Pooling works  
    - Timeouts handled  
    - Reconnection works  
}
```

TEST-E2E-033: Database Migration

```
func TestDatabaseMigration(t *testing.T) {  
    // Given: Database with existing schema  
    // When: Migration applied  
    // Then: Schema updated correctly  
  
    Steps:  
    1. Create initial schema  
    2. Define migration  
    3. Apply migration  
    4. Verify schema changes  
    5. Test rollback  
  
    Assertions:
```

- Migration applies
- Schema correct
- Data preserved
- Rollback works

```
}
```

TEST-E2E-034: Database Transaction Handling

```
func TestDatabaseTransactions(t *testing.T) {  
    // Given: Database operations  
    // When: Transactions used  
    // Then: ACID properties maintained  
  
    Steps:  
    1. Begin transaction  
    2. Perform operations  
    3. Test commit  
    4. Test rollback  
    5. Verify isolation  
  
    Assertions:  
    - Transactions work  
    - Commit succeeds  
    - Rollback works  
    - Isolation maintained  
}
```

TEST-E2E-035: Database Backup and Recovery

```
func TestDatabaseBackupRecovery(t *testing.T) {  
    // Given: Database with data  
    // When: Backup created and restored  
    // Then: Data recovered correctly  
  
    Steps:  
    1. Insert test data  
    2. Create backup  
    3. Modify data  
    4. Restore backup  
    5. Verify data integrity  
  
    Assertions:  
    - Backup created  
    - Data modified
```

```
- Restore successful
- Integrity verified
}
```

TEST-E2E-036: Database Performance

```
func TestDatabasePerformance(t *testing.T) {
    // Given: Database with data
    // When: High-volume operations performed
    // Then: Performance acceptable

    Steps:
    1. Load test data
    2. Execute queries
    3. Monitor performance
    4. Test indexes
    5. Optimize if needed

    Assertions:
    - Queries performant
    - Indexes effective
    - No bottlenecks
    - Optimization helps
}
```

3. Redis Integration (4 Tests)

TEST-E2E-037: Redis Connection and Basic Operations

```
func TestRedisConnection(t *testing.T) {
    // Given: Redis server available
    // When: Redis operations performed
    // Then: Operations succeed

    Steps:
    1. Connect to Redis
    2. Perform basic operations
    3. Test different data types
    4. Verify persistence
    5. Check reconnection

    Assertions:
    - Connection works
    - Operations succeed
}
```

- Data types supported
- Reconnection works

```
}
```

TEST-E2E-038: Redis Caching

```
func TestRedisCaching(t *testing.T) {  
    // Given: Application with caching  
    // When: Cache operations performed  
    // Then: Caching works correctly
```

Steps:

1. Configure caching
2. Cache data
3. Retrieve cached data
4. Test cache expiration
5. Handle cache misses

Assertions:

- Data cached
- Retrieval works
- Expiration functions
- Misses handled

```
}
```

TEST-E2E-039: Redis Session Management

```
func TestRedisSessionManagement(t *testing.T) {  
    // Given: Session-based application  
    // When: Sessions stored in Redis  
    // Then: Sessions managed correctly
```

Steps:

1. Configure session storage
2. Create sessions
3. Retrieve sessions
4. Test expiration
5. Handle concurrency

Assertions:

- Sessions stored
- Retrieval works
- Expiration handled

```
- Concurrency safe
}
```

TEST-E2E-040: Redis Pub/Sub

```
func TestRedisPubSub(t *testing.T) {
    // Given: Redis pub/sub configured
    // When: Messages published/subscribed
    // Then: Communication works
```

Steps:

1. Set up pub/sub
2. Subscribe to channels
3. Publish messages
4. Receive messages
5. Handle patterns

Assertions:

- Subscriptions work
- Publishing succeeds
- Messages received
- Patterns supported

```
}
```

4. SSH Worker Integration (5 Tests)

TEST-E2E-041: SSH Worker Connection

```
func TestSSHWorkerConnection(t *testing.T) {
    // Given: SSH workers configured
    // When: Connection established
    // Then: Connection secure and functional
```

Steps:

1. Configure SSH credentials
2. Establish connection
3. Verify authentication
4. Test secure channels
5. Handle disconnections

Assertions:

- Connection secure
- Authentication works
- Channels established

```
    - Disconnection handled
}
```

TEST-E2E-042: Worker Pool Management

```
func TestWorkerPoolManagement(t *testing.T) {
    // Given: Multiple SSH workers
    // When: Pool managed dynamically
    // Then: Pool functions correctly

    Steps:
    1. Register workers
    2. Monitor health
    3. Distribute tasks
    4. Handle failures
    5. Scale pool

    Assertions:
    - Workers registered
    - Health monitored
    - Tasks distributed
    - Scaling works
}
```

TEST-E2E-043: Task Distribution

```
func TestTaskDistribution(t *testing.T) {
    // Given: Worker pool with tasks
    // When: Tasks distributed
    // Then: Distribution optimal

    Steps:
    1. Create tasks
    2. Assess worker capabilities
    3. Distribute tasks
    4. Monitor execution
    5. Balance load

    Assertions:
    - Tasks created
    - Capabilities assessed
    - Distribution fair
    - Load balanced
}
```

TEST-E2E-044: Worker Health Monitoring

```
func TestWorkerHealthMonitoring(t *testing.T) {  
    // Given: Active worker pool  
    // When: Health checks performed  
    // Then: Health status accurate  
  
    Steps:  
    1. Establish health checks  
    2. Monitor healthy workers  
    3. Simulate failures  
    4. Detect unhealthy workers  
    5. Handle recovery  
  
    Assertions:  
    - Checks run regularly  
    - Health accurate  
    - Failures detected  
    - Recovery handled  
}
```

TEST-E2E-045: Cross-Platform Worker Compatibility

```
func TestCrossPlatformWorkerCompatibility(t *testing.T) {  
    // Given: Workers on different platforms  
    // When: Tasks executed across platforms  
    // Then: Compatibility maintained  
  
    Steps:  
    1. Set up multi-platform workers  
    2. Create platform-agnostic tasks  
    3. Distribute across platforms  
    4. Verify execution  
    5. Handle platform specifics  
  
    Assertions:  
    - Workers connect  
    - Tasks compatible  
    - Execution successful  
    - Platform differences handled  
}
```

5. Notification System (5 Tests)

TEST-E2E-046: Multi-Channel Notifications

```
func TestMultiChannelNotifications(t *testing.T) {  
    // Given: Multiple notification channels  
    // When: Notifications sent  
    // Then: All channels receive notifications
```

Steps:

1. Configure channels (email, Slack, Discord, Telegram)
2. Create notification events
3. Send notifications
4. Verify delivery
5. Check content formatting

Assertions:

- All channels configured
- Notifications delivered
- Content formatted
- Delivery confirmed

```
}
```

TEST-E2E-047: Notification Templates

```
func TestNotificationTemplates(t *testing.T) {  
    // Given: Notification system  
    // When: Templates used  
    // Then: Templates render correctly
```

Steps:

1. Create templates
2. Populate with data
3. Render notifications
4. Customize templates
5. Test conditional logic

Assertions:

- Templates created
- Data populated
- Rendering works
- Customization possible

```
}
```

TEST-E2E-048: Notification Scheduling

```
func TestNotificationScheduling(t *testing.T) {  
    // Given: Notification system  
    // When: Notifications scheduled  
    // Then: Delivered at correct time  
  
    Steps:  
    1. Schedule notifications  
    2. Set delivery times  
    3. Wait for delivery  
    4. Verify timing  
    5. Handle time zones  
  
    Assertions:  
    - Scheduling works  
    - Timing accurate  
    - Time zones handled  
    - Delivery confirmed  
}
```

TEST-E2E-049: Notification Preferences

```
func TestNotificationPreferences(t *testing.T) {  
    // Given: Users with preferences  
    // When: Preferences configured  
    // Then: Notifications respect preferences  
  
    Steps:  
    1. Set user preferences  
    2. Configure channels  
    3. Send notifications  
    4. Verify filtering  
    5. Test do-not-disturb  
  
    Assertions:  
    - Preferences saved  
    - Filtering works  
    - Channels respected  
    - Do-not-disturb honored  
}
```

TEST-E2E-050: Notification Error Handling

```
func TestNotificationErrorHandling(t *testing.T) {  
    // Given: Notification system  
    // When: Delivery failures occur  
    // Then: Errors handled gracefully  
  
    Steps:  
    1. Configure notifications  
    2. Simulate failures  
    3. Handle retries  
    4. Log errors  
    5. Alert administrators  
  
    Assertions:  
    - Failures detected  
    - Retries attempted  
    - Errors logged  
    - Admin alerted  
}
```

6. Memory System Integration (5 Tests)

TEST-E2E-051: Memory Provider Switching

```
func TestMemoryProviderSwitching(t *testing.T) {  
    // Given: Multiple memory providers  
    // When: Provider switched  
    // Then: Switching seamless  
  
    Steps:  
    1. Configure multiple providers  
    2. Store data in one  
    3. Switch providers  
    4. Verify data access  
    5. Test migration  
  
    Assertions:  
    - Providers switch  
    - Data accessible  
    - Migration works  
    - No data loss  
}
```

TEST-E2E-052: Long-term Memory Persistence

```
func TestLongTermMemoryPersistence(t *testing.T) {  
    // Given: Conversations and context  
    // When: Stored in memory system  
    // Then: Memory persists and retrievable  
  
    Steps:  
    1. Have conversation  
    2. Store in memory  
    3. Retrieve later  
    4. Verify context  
    5. Test search  
  
    Assertions:  
    - Memory stored  
    - Retrieval works  
    - Context preserved  
    - Search functions  
}
```

TEST-E2E-053: Memory Search and Retrieval

```
func TestMemorySearchRetrieval(t *testing.T) {  
    // Given: Stored memories  
    // When: Search performed  
    // Then: Relevant memories found  
  
    Steps:  
    1. Store various memories  
    2. Search by keywords  
    3. Filter by metadata  
    4. Rank by relevance  
    5. Retrieve details  
  
    Assertions:  
    - Search works  
    - Filtering functions  
    - Relevance ranking  
    - Details retrieved  
}
```

TEST-E2E-054: Memory System Performance

```
func TestMemorySystemPerformance(t *testing.T) {  
    // Given: Large memory dataset  
    // When: Operations performed  
    // Then: Performance acceptable  
  
    Steps:  
    1. Load large dataset  
    2. Perform operations  
    3. Measure response times  
    4. Test concurrent access  
    5. Optimize if needed  
  
    Assertions:  
    - Performance measured  
    - Response times good  
    - Concurrency handled  
    - Optimization effective  
}
```

TEST-E2E-055: Memory System Backup/Recovery

```
func TestMemorySystemBackupRecovery(t *testing.T) {  
    // Given: Memory system with data  
    // When: Backup created and restored  
    // Then: Data recovered correctly  
  
    Steps:  
    1. Store memories  
    2. Create backup  
    3. Simulate data loss  
    4. Restore from backup  
    5. Verify integrity  
  
    Assertions:  
    - Backup created  
    - Data loss simulated  
    - Restore successful  
    - Integrity verified  
}
```

DISTRIBUTED SYSTEM TESTS (20 Tests)

1. Multi-Worker Task Distribution (5 Tests)

TEST-E2E-056: Task Queue Management

```
func TestTaskQueueManagement(t *testing.T) {  
    // Given: Multiple tasks and workers  
    // When: Tasks queued for execution  
    // Then: Queue managed efficiently
```

Steps:

1. Create task queue
2. Add tasks with priorities
3. Assign to workers
4. Monitor execution
5. Handle failures

Assertions:

- Queue created
- Priorities respected
- Tasks assigned
- Failures handled

```
}
```

TEST-E2E-057: Load Balancing Algorithms

```
func TestLoadBalancingAlgorithms(t *testing.T) {  
    // Given: Multiple workers with different capacities  
    // When: Tasks need distribution  
    // Then: Load balanced optimally
```

Steps:

1. Configure workers
2. Set capacities
3. Generate tasks
4. Apply balancing
5. Verify distribution

Assertions:

- Workers configured
- Capacities set
- Balancing applied

```
- Distribution optimal
}
```

TEST-E2E-058: Task Priority Handling

```
func TestTaskPriorityHandling(t *testing.T) {
    // Given: Tasks with different priorities
    // When: Tasks queued for execution
    // Then: Priorities respected

    Steps:
    1. Create tasks with priorities
    2. Queue tasks
    3. Monitor execution order
    4. Test priority changes
    5. Verify starvation prevention

    Assertions:
    - Priorities set
    - Order correct
    - Changes handled
    - No starvation
}
```

TEST-E2E-059: Task Dependencies

```
func TestTaskDependencies(t *testing.T) {
    // Given: Tasks with dependencies
    // When: Tasks scheduled for execution
    // Then: Dependencies respected

    Steps:
    1. Create dependent tasks
    2. Define dependencies
    3. Schedule execution
    4. Verify order
    5. Handle failures

    Assertions:
    - Dependencies defined
    - Order respected
    - Execution correct
    - Failures handled
}
```

TEST-E2E-060: Task Result Aggregation

```
func TestTaskResultAggregation(t *testing.T) {  
    // Given: Distributed task execution  
    // When: Results need aggregation  
    // Then: Results combined correctly  
  
    Steps:  
    1. Distribute tasks  
    2. Execute in parallel  
    3. Collect results  
    4. Aggregate results  
    5. Verify completeness  
  
    Assertions:  
    - Tasks distributed  
    - Results collected  
    - Aggregation correct  
    - Completeness verified  
}
```

2. Load Balancing & Failover (5 Tests)

TEST-E2E-061: Dynamic Load Balancing

```
func TestDynamicLoadBalancing(t *testing.T) {  
    // Given: Changing load conditions  
    // When: Load balancing adapts  
    // Then: Performance maintained  
  
    Steps:  
    1. Establish baseline load  
    2. Monitor performance  
    3. Increase load gradually  
    4. Observe balancing  
    5. Verify performance  
  
    Assertions:  
    - Baseline established  
    - Monitoring active  
    - Balancing adaptive  
    - Performance maintained  
}
```

TEST-E2E-062: Worker Failure Detection

```
func TestWorkerFailureDetection(t *testing.T) {  
    // Given: Active worker pool  
    // When: Worker fails  
    // Then: Failure detected quickly  
  
    Steps:  
    1. Monitor healthy workers  
    2. Simulate worker failure  
    3. Detect failure  
    4. Isolate failed worker  
    5. Alert administrators  
  
    Assertions:  
    - Health monitoring  
    - Failure detected  
    - Isolation quick  
    - Alert sent  
}
```

TEST-E2E-063: Automatic Failover

```
func TestAutomaticFailover(t *testing.T) {  
    // Given: Worker failure detected  
    // When: Failover triggered  
    // Then: Service continues  
  
    Steps:  
    1. Create redundant setup  
    2. Simulate primary failure  
    3. Trigger failover  
    4. Verify backup active  
    5. Test functionality  
  
    Assertions:  
    - Redundancy exists  
    - Failover triggers  
    - Backup activates  
    - Functionality preserved  
}
```

TEST-E2E-064: Circuit Breaker Pattern

```
func TestCircuitBreakerPattern(t *testing.T) {  
    // Given: Failing service  
    // When: Circuit breaker activated  
    // Then: System protected  
  
    Steps:  
    1. Monitor service health  
    2. Simulate failures  
    3. Trigger circuit breaker  
    4. Verify fallback  
    5. Test recovery  
  
    Assertions:  
    - Monitoring active  
    - Breaker triggers  
    - Fallback works  
    - Recovery detected  
}
```

TEST-E2E-065: Graceful Degradation

```
func TestGracefulDegradation(t *testing.T) {  
    // Given: System under stress  
    // When: Resources constrained  
    // Then: Service degrades gracefully  
  
    Steps:  
    1. Establish normal operation  
    2. Introduce constraints  
    3. Monitor degradation  
    4. Verify core functions  
    5. Test recovery  
  
    Assertions:  
    - Normal operation  
    - Constraints introduced  
    - Graceful degradation  
    - Core functions work  
}
```

3. Network Partition Recovery (5 Tests)

TEST-E2E-066: Network Partition Detection

```
func TestNetworkPartitionDetection(t *testing.T) {  
    // Given: Distributed system  
    // When: Network partition occurs  
    // Then: Partition detected quickly  
  
    Steps:  
    1. Establish cluster  
    2. Simulate partition  
    3. Detect isolation  
    4. Verify quorum  
    5. Alert on partition  
  
    Assertions:  
    - Cluster stable  
    - Partition detected  
    - Isolation verified  
    - Alert generated  
}
```

TEST-E2E-067: Split-Brain Prevention

```
func TestSplitBrainPrevention(t *testing.T) {  
    // Given: Network partition  
    // When: Multiple partitions form  
    // Then: Split-brain prevented  
  
    Steps:  
    1. Create partition scenario  
    2. Monitor cluster state  
    3. Verify quorum rules  
    4. Prevent dual masters  
    5. Maintain consistency  
  
    Assertions:  
    - Partition created  
    - Quorum maintained  
    - Single authority  
    - Consistency preserved  
}
```

TEST-E2E-068: Partition Recovery

```
func TestPartitionRecovery(t *testing.T) {  
    // Given: Network partition  
    // When: Network heals  
    // Then: System recovers completely  
  
    Steps:  
    1. Simulate partition  
    2. Operate independently  
    3. Heal partition  
    4. Reconcile state  
    5. Verify consistency  
  
    Assertions:  
    - Partition simulated  
    - Independent operation  
    - Healing detected  
    - State reconciled  
}
```

TEST-E2E-069: Data Consistency During Partition

```
func TestDataConsistencyDuringPartition(t *testing.T) {  
    // Given: Partitioned cluster  
    // When: Data modified in partitions  
    // Then: Consistency maintained/recovered  
  
    Steps:  
    1. Create partition  
    2. Modify data in partitions  
    3. Track changes  
    4. Heal partition  
    5. Reconcile conflicts  
  
    Assertions:  
    - Partition created  
    - Changes tracked  
    - Conflicts detected  
    - Reconciliation correct  
}
```

TEST-E2E-070: Quorum Maintenance

```
func TestQuorumMaintenance(t *testing.T) {  
    // Given: Cluster with quorum  
    // When: Nodes leave/join  
    // Then: Quorum maintained  
  
    Steps:  
    1. Establish quorum  
    2. Remove nodes  
    3. Verify quorum lost  
    4. Add new nodes  
    5. Restore quorum  
  
    Assertions:  
    - Quorum established  
    - Loss detected  
    - New nodes added  
    - Quorum restored  
}
```

4. Concurrent User Sessions (5 Tests)

TEST-E2E-071: Session Isolation

```
func TestSessionIsolation(t *testing.T) {  
    // Given: Multiple user sessions  
    // When: Sessions active simultaneously  
    // Then: Isolation maintained  
  
    Steps:  
    1. Create multiple sessions  
    2. Perform operations  
    3. Verify isolation  
    4. Test resource sharing  
    5. Check security boundaries  
  
    Assertions:  
    - Sessions isolated  
    - Operations independent  
    - Resources separated  
    - Security maintained  
}
```

TEST-E2E-072: Resource Allocation

```
func TestResourceAllocation(t *testing.T) {  
    // Given: Limited system resources  
    // When: Multiple sessions compete  
    // Then: Allocation fair and efficient
```

Steps:

1. Establish resource limits
2. Create competing sessions
3. Monitor allocation
4. Test fairness
5. Handle exhaustion

Assertions:

- Limits enforced
- Allocation fair
- Competition handled
- Exhaustion graceful

```
}
```

TEST-E2E-073: Session Timeout Handling

```
func TestSessionTimeoutHandling(t *testing.T) {  
    // Given: Active user sessions  
    // When: Sessions timeout  
    // Then: Timeout handled gracefully
```

Steps:

1. Configure timeouts
2. Create active sessions
3. Wait **for** timeout
4. Handle expiration
5. Clean up resources

Assertions:

- Timeouts configured
- Expiration detected
- Handling graceful
- Cleanup complete

```
}
```

TEST-E2E-074: Concurrent Data Access

```
func TestConcurrentDataAccess(t *testing.T) {  
    // Given: Shared data resources  
    // When: Concurrent access attempted  
    // Then: Access synchronized correctly  
  
    Steps:  
    1. Identify shared resources  
    2. Create concurrent access  
    3. Monitor synchronization  
    4. Detect conflicts  
    5. Resolve deadlocks  
  
    Assertions:  
    - Access synchronized  
    - Conflicts detected  
    - Deadlocks prevented  
    - Consistency maintained  
}
```

TEST-E2E-075: Session Migration

```
func TestSessionMigration(t *testing.T) {  
    // Given: Active session on one node  
    // When: Session migrated to another node  
    // Then: Migration seamless  
  
    Steps:  
    1. Create active session  
    2. Initiate migration  
    3. Transfer state  
    4. Verify continuity  
    5. Clean up original  
  
    Assertions:  
    - Migration initiated  
    - State transferred  
    - Continuity maintained  
    - Cleanup performed  
}
```

PLATFORM-SPECIFIC TESTS (15 Tests)

1. Linux Deployment (3 Tests)

TEST-E2E-076: Linux Package Installation

```
func TestLinuxPackageInstallation(t *testing.T) {  
    // Given: Linux system (Ubuntu/Debian)  
    // When: HelixCode installed  
    // Then: Installation successful  
  
    Steps:  
    1. Download package  
    2. Install dependencies  
    3. Install HelixCode  
    4. Verify installation  
    5. Test functionality  
  
    Assertions:  
    - Package downloads  
    - Dependencies met  
    - Installation completes  
    - Functionality verified  
}
```

TEST-E2E-077: Linux Service Management

```
func TestLinuxServiceManagement(t *testing.T) {  
    // Given: HelixCode installed on Linux  
    // When: Managed as system service  
    // Then: Service operations work  
  
    Steps:  
    1. Create service definition  
    2. Start service  
    3. Check status  
    4. Stop service  
    5. Enable auto-start  
  
    Assertions:  
    - Service created  
    - Start successful  
    - Status correct
```

```
- Auto-start works
}
```

TEST-E2E-078: Linux Security Configuration

```
func TestLinuxSecurityConfiguration(t *testing.T) {
    // Given: HelixCode on Linux
    // When: Security configured
    // Then: System secure

    Steps:
    1. Configure firewall
    2. Set file permissions
    3. Configure SELinux/AppArmor
    4. Set up monitoring
    5. Test security

    Assertions:
    - Firewall configured
    - Permissions set
    - MAC configured
    - Monitoring active
}
```

2. macOS Deployment (3 Tests)

TEST-E2E-079: macOS Application Bundle

```
func TestMacOSApplicationBundle(t *testing.T) {
    // Given: macOS system
    // When: Application bundle created
    // Then: Bundle works correctly

    Steps:
    1. Create app bundle
    2. Sign application
    3. Test installation
    4. Verify permissions
    5. Check functionality

    Assertions:
    - Bundle created
    - Signing successful
    - Installation works
}
```

```
    - Functionality verified  
}
```

TEST-E2E-080: macOS Code Signing

```
func TestMacOSCodeSigning(t *testing.T) {  
    // Given: macOS application  
    // When: Code signing applied  
    // Then: Signing valid  
  
    Steps:  
    1. Generate certificates  
    2. Sign application  
    3. Verify signature  
    4. Test notarization  
    5. Validate gatekeeper  
  
    Assertions:  
    - Certificates valid  
    - Signing successful  
    - Notarization passes  
    - Gatekeeper approves  
}
```

TEST-E2E-081: macOS System Integration

```
func TestMacOSSystemIntegration(t *testing.T) {  
    // Given: HelixCode on macOS  
    // When: System integration configured  
    // Then: Integration works  
  
    Steps:  
    1. Create launchd service  
    2. Configure notifications  
    3. Set up file associations  
    4. Test system calls  
    5. Verify integration  
  
    Assertions:  
    - Service created  
    - Notifications work  
    - Associations set  
    - Integration verified  
}
```

3. Windows Integration (3 Tests)

TEST-E2E-082: Windows WSL Integration

```
func TestWindowsWSLIntegration(t *testing.T) {  
    // Given: Windows with WSL  
    // When: HelixCode runs in WSL  
    // Then: Integration seamless  
  
    Steps:  
    1. Install WSL  
    2. Set up Linux environment  
    3. Install HelixCode  
    4. Test Windows integration  
    5. Verify file sharing  
  
    Assertions:  
    - WSL installed  
    - Environment ready  
    - Integration works  
    - Sharing functional  
}
```

TEST-E2E-083: Windows Service Installation

```
func TestWindowsServiceInstallation(t *testing.T) {  
    // Given: Windows system  
    // When: HelixCode installed as service  
    // Then: Service works correctly  
  
    Steps:  
    1. Create service  
    2. Install service  
    3. Start service  
    4. Test functionality  
    5. Manage service  
  
    Assertions:  
    - Service created  
    - Installation successful  
    - Functionality verified  
    - Management works  
}
```

TEST-E2E-084: Windows Registry Configuration

```
func TestWindowsRegistryConfiguration(t *testing.T) {  
    // Given: HelixCode on Windows  
    // When: Registry configuration needed  
    // Then: Registry handled correctly  
  
    Steps:  
    1. Create registry keys  
    2. Set configuration values  
    3. Test reading/writing  
    4. Handle permissions  
    5. Clean up properly  
  
    Assertions:  
    - Keys created  
    - Values set  
    - Access works  
    - Cleanup complete  
}
```

4. Container Technologies (3 Tests)

TEST-E2E-085: Docker Containerization

```
func TestDockerContainerization(t *testing.T) {  
    // Given: HelixCode application  
    // When: Containerized with Docker  
    // Then: Container works correctly  
  
    Steps:  
    1. Create Dockerfile  
    2. Build image  
    3. Run container  
    4. Test functionality  
    5. Verify isolation  
  
    Assertions:  
    - Image builds  
    - Container runs  
    - Functionality verified  
    - Isolation maintained  
}
```

TEST-E2E-086: Docker Compose Multi-Service

```
func TestDockerComposeMultiService(t *testing.T) {  
    // Given: Multi-service application  
    // When: Orchestrated with Docker Compose  
    // Then: Services work together  
  
    Steps:  
    1. Create compose file  
    2. Define services  
    3. Start services  
    4. Test integration  
    5. Scale services  
  
    Assertions:  
    - Services defined  
    - Integration works  
    - Communication functional  
    - Scaling effective  
}
```

TEST-E2E-087: Kubernetes Deployment

```
func TestKubernetesDeployment(t *testing.T) {  
    // Given: Containerized application  
    // When: Deployed to Kubernetes  
    // Then: Deployment successful  
  
    Steps:  
    1. Create manifests  
    2. Deploy to cluster  
    3. Verify pods running  
    4. Test service discovery  
    5. Scale deployment  
  
    Assertions:  
    - Manifests valid  
    - Deployment successful  
    - Services discoverable  
    - Scaling works  
}
```

5. Mobile Platforms (3 Tests)

TEST-E2E-088: iOS App Integration

```
func TestIOSAppIntegration(t *testing.T) {  
    // Given: iOS application  
    // When: Integrated with HelixCode  
    // Then: Integration works  
  
    Steps:  
    1. Build iOS app  
    2. Configure API integration  
    3. Test communication  
    4. Verify data sync  
    5. Test offline mode  
  
    Assertions:  
    - App builds  
    - API works  
    - Sync functional  
    - Offline supported  
}
```

TEST-E2E-089: Android App Integration

```
func TestAndroidAppIntegration(t *testing.T) {  
    // Given: Android application  
    // When: Integrated with HelixCode  
    // Then: Integration successful  
  
    Steps:  
    1. Build Android app  
    2. Set up integration  
    3. Test API calls  
    4. Verify functionality  
    5. Test across devices  
  
    Assertions:  
    - App built  
    - Integration works  
    - API functional  
    - Device compatibility  
}
```

TEST-E2E-090: Mobile API Security

```
func TestMobileAPISecurity(t *testing.T) {  
    // Given: Mobile app integration  
    // When: API calls made from mobile  
    // Then: Security maintained  
  
    Steps:  
    1. Configure mobile auth  
    2. Test token handling  
    3. Verify encryption  
    4. Test certificate pinning  
    5. Validate requests  
  
    Assertions:  
    - Auth secure  
    - Tokens handled  
    - Encryption active  
    - Requests validated  
}
```

TEST IMPLEMENTATION PRIORITIES

Phase 1A: Critical Core Workflows (Week 2)

Priority 1 (Days 1-2): Authentication & Project Management

- TEST-E2E-001 to TEST-E2E-007 (7 tests)
- These are fundamental to all other operations

Priority 2 (Days 3-4): File Operations & Code Generation

- TEST-E2E-008 to TEST-E2E-015 (8 tests)
- Core functionality for development workflows

Priority 3 (Days 5-6): Build, Test & Debug

- TEST-E2E-016 to TEST-E2E-022 (7 tests)
- Automation and development tools

Priority 4 (Day 7): Configuration & Integration Prep

- TEST-E2E-023 to TEST-E2E-025 (3 tests)
- System configuration and prep for integration tests

Phase 1B: Integration Tests (Week 3)

Priority 1 (Days 1-2): LLM Provider Integration

- TEST-E2E-026 to TEST-E2E-031 (6 tests)
- Core AI functionality

Priority 2 (Days 3-4): Database & Redis

- TEST-E2E-032 to TEST-E2E-040 (9 tests)
- Data persistence and caching

Priority 3 (Days 5-6): SSH Workers & Notifications

- TEST-E2E-041 to TEST-E2E-050 (10 tests)
- Distributed computing and communication

Priority 4 (Day 7): Memory System

- TEST-E2E-051 to TEST-E2E-055 (5 tests)
- Advanced AI memory features

Phase 1C: Distributed & Platform Tests (Week 4)

Priority 1 (Days 1-2): Task Distribution & Load Balancing

- TEST-E2E-056 to TEST-E2E-065 (10 tests)
- Core distributed functionality

Priority 2 (Days 3-4): Network & Sessions

- TEST-E2E-066 to TEST-E2E-075 (10 tests)
- Network resilience and user management

Priority 3 (Days 5-7): Platform-Specific

- TEST-E2E-076 to TEST-E2E-090 (15 tests)
- Cross-platform compatibility

TEST IMPLEMENTATION FRAMEWORK

Test Structure Template:

```
func TestXXXX(t *testing.T) {  
    // Setup  
    ctx := context.Background()  
    server := setupTestServer(t)  
    defer server.Cleanup()  
  
    // Given
```

```

testData := prepareTestData(t)

// When
result, err := performOperation(ctx, testData)

// Then
require.NoError(t, err)
assert.Equal(t, expectedResult, result)

// Cleanup
cleanupTestData(t, testData)
}

```

Test Data Management:

```

// Use factories for test data
type TestDataFactory struct {
    // Factories for users, projects, etc.
}

// Clean up after each test
func cleanupTestData(t *testing.T, data interface{}) {
    // Remove test data from database
    // Clean up files
    // Reset mocks
}

```

Test Environment Setup:

```

func setupTestServer(t *testing.T) *TestServer {
    // Start test server
    // Configure test database
    // Set up mock services
    // Return test server instance
}

```

SUCCESS METRICS

Test Implementation Progress:

Week 2 Target: 25 Core Workflow Tests (100%)
 Week 3 Target: 30 Integration Tests (100%)
 Week 4 Target: 20 Distributed + 15 Platform Tests (100%)

Overall Target: 90 new tests implemented
Success Criteria: All tests passing >95% success rate

Quality Gates:

- Gate 1: All tests compile without errors
- Gate 2: Tests are deterministic (no flaky tests)
- Gate 3: Tests have proper cleanup
- Gate 4: Tests cover edge cases
- Gate 5: Tests have meaningful assertions

IMPLEMENTATION NOTES

Critical Dependencies:

- Phase 0 must be complete (build system working)
- Test infrastructure must be functional
- Mock services must be available
- Test databases must be configurable

Resource Requirements:

- Test environment with multiple platforms
- Database instances for testing
- Redis instance for caching tests
- SSH access to test workers
- Multiple LLM provider accounts

Timeline:

- **Week 2:** Core workflows (25 tests)
- **Week 3:** Integration tests (30 tests)
- **Week 4:** Distributed & platform (35 tests)
- **Total:** 90 tests in 3 weeks

Status: **READY FOR IMPLEMENTATION** - Framework exists, 90 tests to be created

Next: Begin Phase 1A with core workflow tests

Dependencies: Phase 0 completion (build system fixed)

Specification created: December 11, 2025 - Ready for implementation phase